**Descripción de atributos y métodos**

**Objeto del documento**

Se describen los atributos y los métodos de las clases visibles en el nuevo diagrama de clases del sistema (Proyecto IS). La documentación sigue una estructura organizada por módulos según las clases y gestores presentes en el modelo aportado.

**Criterio de trazabilidad:** los atributos ya dibujados se conservan con sus tipos; los campos que faltan para que una funcionalidad sea viable se identifican expresamente como «propuestos». Las operaciones son un diseño coherente con las relaciones y nombres de las clases del diagrama.

**Notación:** <<->> denota atributo privado, <<+>> método público; Date representa el tipo date y Vector<T> el contenedor representado en las clases Lista.

## 1. Clases del Núcleo y Datos

### 1.1. Medición

Representa los valores registrados para una medición en el sistema.

- **Atributos:**
  - presion: double — Valor de presión asociado a la medición.
  - inclinacion: double — Valor de inclinación asociado a la medición.
  - fechaHora: Date [propuesto] — Momento de toma de la medida; necesario para organizar un histórico cronológico.
- **Métodos:**
  - + obtenerPresion(): double — Devuelve la presión registrada.
  - + obtenerInclinacion(): double — Devuelve la inclinación registrada.
  - + actualizarValores(presion: double, inclinacion: double): void — Actualiza los dos valores cuando la operación esté autorizada.
  - + validarDatos(): boolean — Comprueba que los campos numéricos contienen datos utilizables.

### 1.2. Incidencias

Representa una incidencia individual registrada.

- **Atributos:**
  - solucionada: boolean — Indica si la incidencia está resuelta.
  - fechaSuceso: Date [propuesto] — Fecha en la que ocurre o se detecta la incidencia.
  - descripcion: String [propuesto] — Explica el hecho comunicado.
- **Métodos:**
  - + registrarDescripcion(texto: String): void — Guarda los detalles de la incidencia.
  - + marcarSolucionada(): void — Establece el estado solucionada a verdadero.
  - + estaSolucionada(): boolean — Consulta el estado de resolución.
  - + consultarFechaSuceso(): Date — Devuelve la fecha del suceso.

### 1.3. Tareas Mantenimiento

Representa una tarea de mantenimiento concreta del sistema.

- **Atributos:**
  - completada: boolean — Indica si la tarea está finalizada.
  - descripcion: String [propuesto] — Identifica el trabajo de mantenimiento pendiente o realizado.
- **Métodos:**
  - + establecerDescripcion(texto: String): void — Describe la actuación que debe realizarse.
  - + marcarCompletada(): void — Cambia el estado de la tarea a completada.
  - + estaCompletada(): boolean — Permite comprobar si se ha finalizado la tarea.

### 1.4. PlanActuacion

Recoge la actuación prevista ante las necesidades detectadas.

- **Atributos:**
  - descripcion: String [propuesto] — Expone las actuaciones previstas.
  - fechaCreacion: Date [propuesto] — Identifica cuándo se estableció el plan.
  - estado: String [propuesto] — Permite consultar su situación.
- **Métodos:**
  - + definirActuacion(descripcion: String): void — Establece el contenido del plan.
  - + modificarActuacion(descripcion: String): void — Actualiza su contenido.
  - + consultarActuacion(): String — Devuelve el contenido del plan.
  - + consultarEstado(): String — Permite conocer su estado.

### 1.5. ZonaPresa

Representa una zona específica de la presa en el modelo.

- **Atributos:**
  - nombre: String [propuesto] — Identifica la zona de la presa.
- **Métodos:**
  - + consultarNombre(): String — Devuelve el nombre de la zona.

## 2. Estructura de Personal (Trabajador y Subclases)

### 2.1. Trabajador

Superclase de los perfiles de personal.

- **Atributos:** Ninguno definido explícitamente en el diagrama.
- **Métodos:**
  - + consultarSistema(): Sistema — Permite acceder al sistema vinculado mediante la asociación correspondiente.

### 2.2. Subclases de Trabajador

El diagrama especializa a Trabajador en **Personal Operativo**, **Encargado**, **Equipo tecnico** y **Direccion**.

- **Personal Operativo:** Encargado de registrar y consultar las lecturas captadas.
    - + tomarMedicion(): Medicion — Recoge la lectura de los sensores y construye una medición.
- **Encargado:** Perfil de supervisión con acceso a la gestión de incidencias y tareas.
    - + consultarIncidencias(): Vector<Incidencias> — Revisa las incidencias documentadas.
    - + consultarTareasPendientes(): Vector<TareasMantenimiento> — Consulta el estado de las tareas.
- **Equipo tecnico:** Personal que consulta y actualiza el estado de las tareas de mantenimiento y resolución.
    - + finalizarTarea(tarea: TareasMantenimiento): void — Marca como completada una tarea.
    - + resolverIncidencia(incidencia: Incidencias): void — Registra la resolución de una incidencia.
- **Direccion:** Perfil vinculado directamente con la gestión de planes de actuación.
    - + elaborarPlanActuacion(descripcion: String): PlanActuacion — Crea un plan y lo vincula.
    - + modificarPlanActuacion(plan: PlanActuacion, descripcion: String): void — Modifica el plan de actuación.

## 3. Sensores y Contratos

### 3.1. Sensor (Interfaz)

Contrato común implementado por los sensores atmosféricos y de inclinación.

- **Atributos:** Ninguno.
- **Métodos:**
    - + leerValor(): double — Devuelve la lectura correspondiente al tipo de sensor.
    - + estaDisponible(): boolean — Indica si el sensor permite obtener una lectura.

### 3.2. SensorAtmosferico y SensorInclinación

Implementaciones concretas del contrato Sensor.

- **Atributos:** Ninguno.
- **Métodos:**
    - + leerValor(): double — Obtiene la lectura específica del sensor correspondiente.
    - + estaDisponible(): boolean — Informa si se puede obtener la lectura.

## 4. Clases Contenedoras (Listas) y Gestores del Sistema

### 4.1. Patrón Común de Listas (**ListaMediciones**, **ListaIncidencias**, **ListaTareasMantenimiento**, **ListaZonaPresa**, **ListaTrabajadores**, **ListaPlanesDeActuacion**)

Clases contenedoras asociadas mediante agregación en el modelo.

- **Métodos comunes:**
    - + agregar(elemento: T): void — Añade un elemento al vector.

- + eliminar(elemento: T): boolean — Quita un elemento si existe.
- + listar(): Vector<T> — Recupera los elementos almacenados.
- + contar(): int — Devuelve cuántos elementos hay.

### 4.2. Clases Gestoras (GestorSistema, GestorSensor, GestorZonaPresa, GestorMediciones, GestorTrabajador, GestorIncidencias, GestorTareasMantenimiento, GestorPlanesDeActuacion)

El diagrama desglosa la lógica de negocio y coordinación en múltiples clases gestoras específicas que actúan sobre sus respectivas listas y elementos.

- **GestorSistema:** Coordina de forma global los subsistemas principales de la aplicación.
- **GestorMediciones:** Coordina la creación, validación y almacenamiento de las mediciones a través de ListaMediciones.
- **GestorIncidencias:** Gestiona el ciclo de vida y filtrado de las incidencias registradas.
- **GestorTareasMantenimiento:** Administra el registro y seguimiento de las tareas de mantenimiento pendientes o completadas.
- **GestorPlanesDeActuacion:** Controla la elaboración, modificación y consulta de los planes de actuación de Dirección.

## 5. Notas y Reglas de Negocio del Modelo

- **Validación previa:** Una medición debe ser validada antes de incorporarse de forma definitiva al sistema.
- **Control de rangos:** Tal como indica la anotación lateral del diagrama original, se ha eliminado la clase independiente «Anomalia» por ser redundante, ya que la detección de anomalías se implementa directamente como una regla de validación o control de rangos sobre los valores de la clase Medicion cuando rebasan los límites normales.